

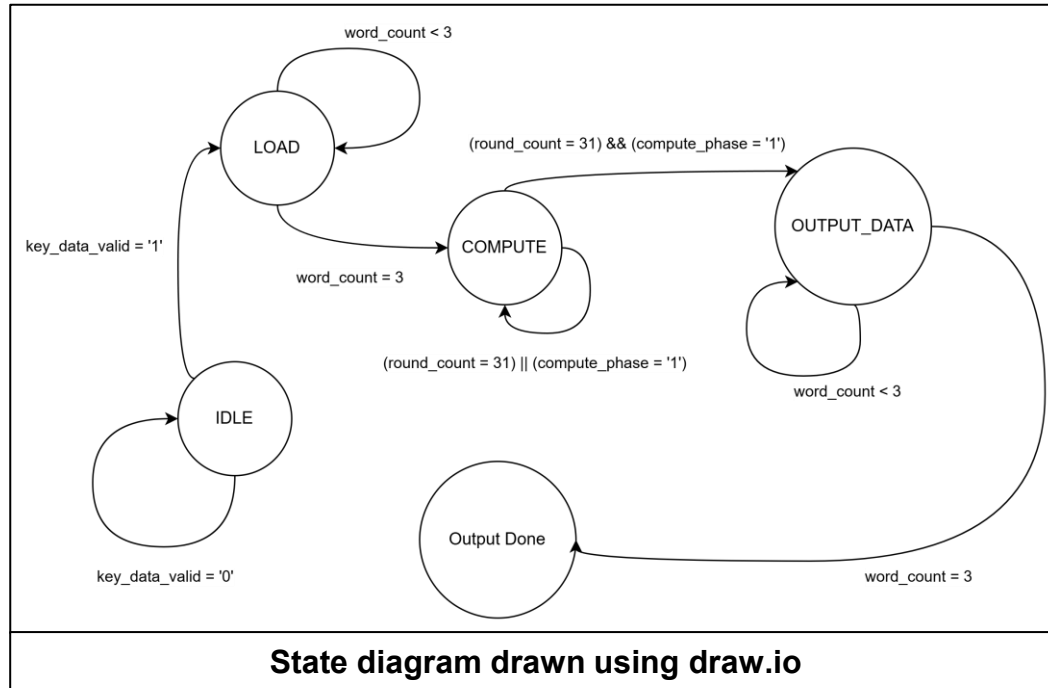


XTEA and Cryptography System

Topics Covered

- VHDL implementation of the XTEA simplex algorithm
- Using UART to validate the XTEA encrypting and decrypting as intended
- Mini router functionality and how it successfully chooses between generated data
- Discussion on how the XTEA algorithm could be implemented within the cryptography system

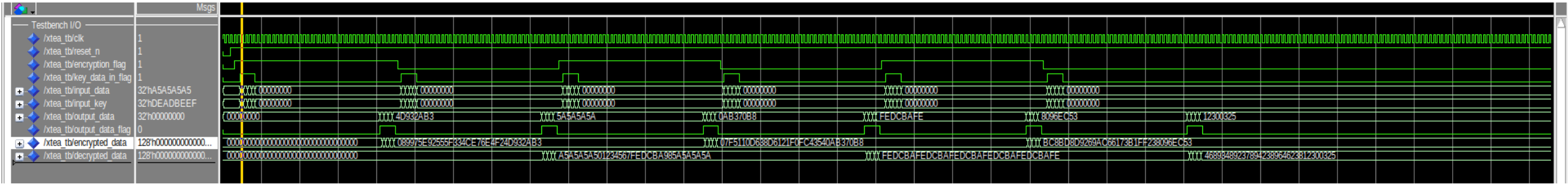
XTEA Encryption Implementation



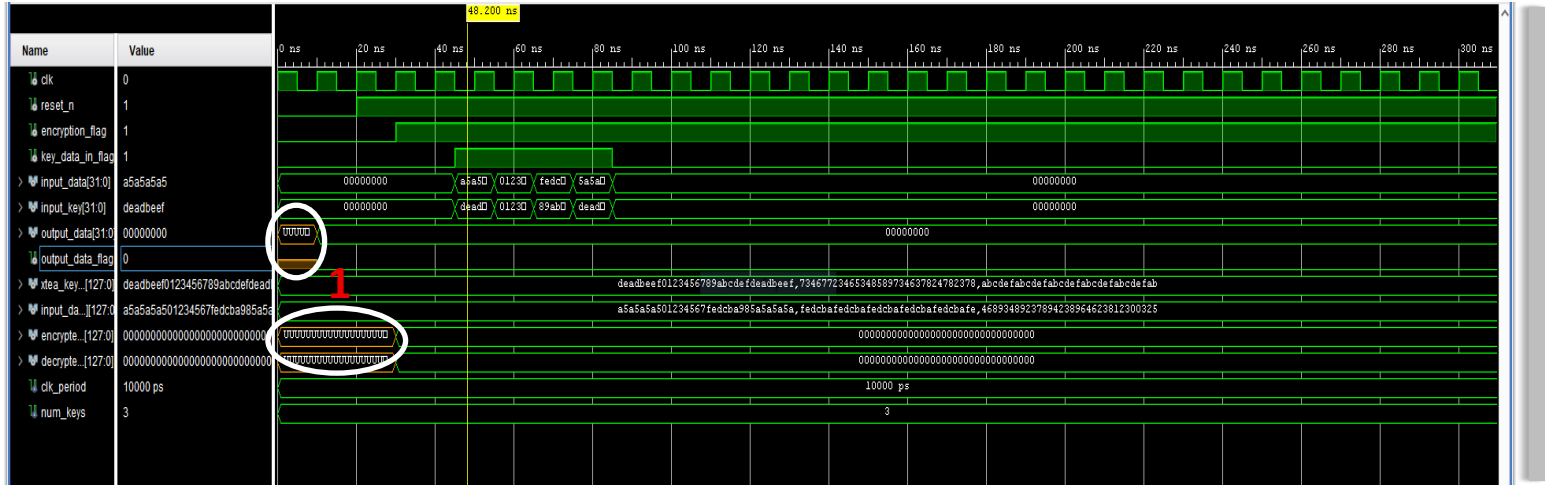
- Simplex version of the XTEA
- Reduced area and power utilization when compared to duplex
- A flag determines if the data should be encrypted or decrypted
- Five state FSM implementation
- Pipelined compute state
- Trade-off between max clock frequency and number of clock cycles.

XTEA in Simulation

Questa Sim Simulation using TCL script



- Data enters in 32-bit intervals, whilst the data_in_flag is high
- The operation of encryption or decryption is decided by encryption_flag



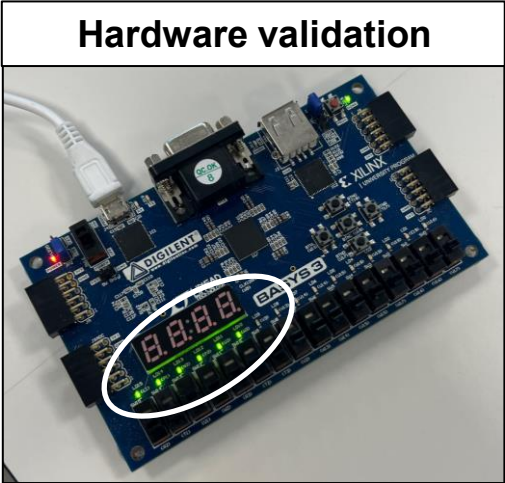
Inference metrics
Inference time of 4475 ns
Number of clock cycles ~ 447
Clock period = 10 ns

XTEA Encryption Hardware Implementation

Power Utilisation [W]	Before Pipelining	After Pipelining
Power	0.091	0.085

Area Utilisation [%]	Before Pipelining	Before Pipelining
LUT	6.03	5.60
FF	2.17	2.48
IO	7.55	7.55

Slack Time	Before Pipelining	After Pipelining
WNS [ns]	-0.244	2.539



Software validation using UART

```
- RESTART: C:\Users\wskg3\F330590_CW_1005\F330590_CW_x
_script\uart_transmit_xtea.py
Connecting to COM15 at 9600 baud
Connected.

ENCRYPT Key Pair 1
Key : DEADBEEF 01234567 89ABCDEF DEADBEEF
Input : ASASASAS 01234567 FEDCBA98 SASASASA
Result : 089975E9 2555F334 CE76E4F2 4D932AB3
Expected: 089975E9 2555F334 CE76E4F2 4D932AB3
RESULT: PASS

DECRYPT Key Pair 1
Key : DEADBEEF 01234567 89ABCDEF DEADBEEF
Input : 089975E9 2555F334 CE76E4F2 4D932AB3
Result : ASASASAS 01234567 FEDCBA98 SASASASA
Expected: ASASASAS 01234567 FEDCBA98 SASASASA
RESULT: PASS

ENCRYPT Key Pair 2
Key : 73467723 46534858 97346378 24782378
Input : FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
Result : 07F5110D 638D6121 F0FC4354 0AB370B8
Expected: 07F5110D 638D6121 F0FC4354 0AB370B8
RESULT: PASS

DECRYPT Key Pair 2
Key : 73467723 46534858 97346378 24782378
Input : 07F5110D 638D6121 F0FC4354 0AB370B8
Result : FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
Expected: FEDCBAFE DCBAFEDC BAFEDCBA FEDCBAFE
RESULT: PASS

ENCRYPT Key Pair 3
Key : ABCDEFAB CDEFABCD EFABCDEF ABCDEFAB
Input : 46893489 23789423 89646238 12300325
Result : BC8BD8D9 269AC661 73B1FF23 8096EC53
Expected: BC8BD8D9 269AC661 73B1FF23 8096EC53
RESULT: PASS

DECRYPT Key Pair 3
Key : ABCDEFAB CDEFABCD EFABCDEF ABCDEFAB
Input : BC8BD8D9 269AC661 73B1FF23 8096EC53
Result : 46893489 23789423 89646238 12300325
Expected: 46893489 23789423 89646238 12300325
RESULT: PASS

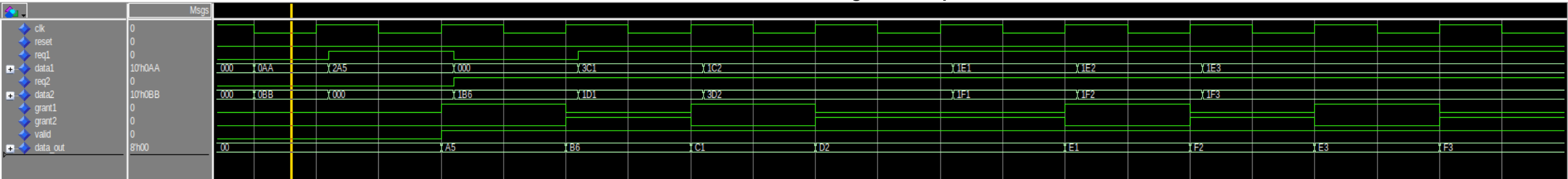
SUMMARY: 6/6 tests passed
PASSED
>>> |
```

Mini Router Implementation

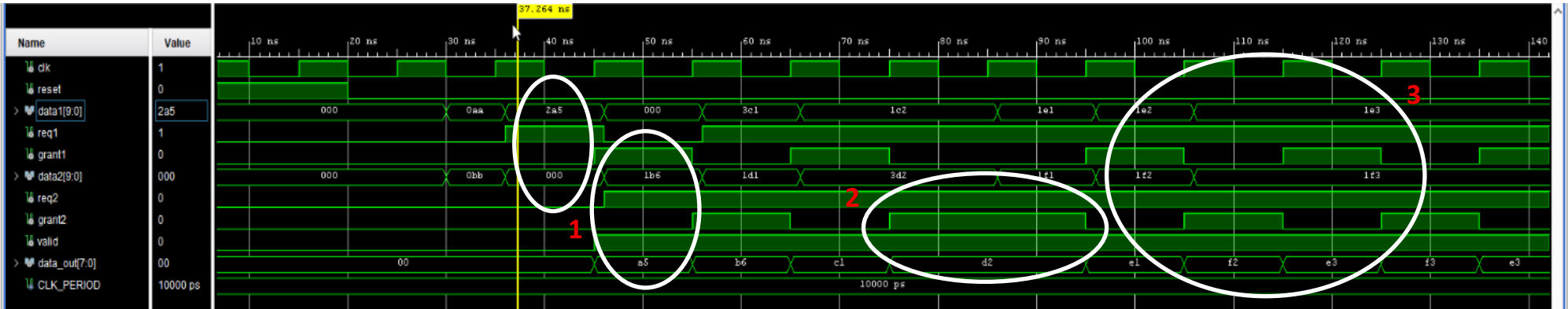
- 10-bit input data, 8 for the payload and 2 for the priority bits
- Active high asynchronous reset
- Singal request → Data passes to output
- Dual request → Data with the higher priority bits [9:8] passes to output
- Equal priority → Resolved using round robin approach, link 1 first followed by link 2 next contention

Mini Router in Simulation

Questa Sim Simulation using TCL Script



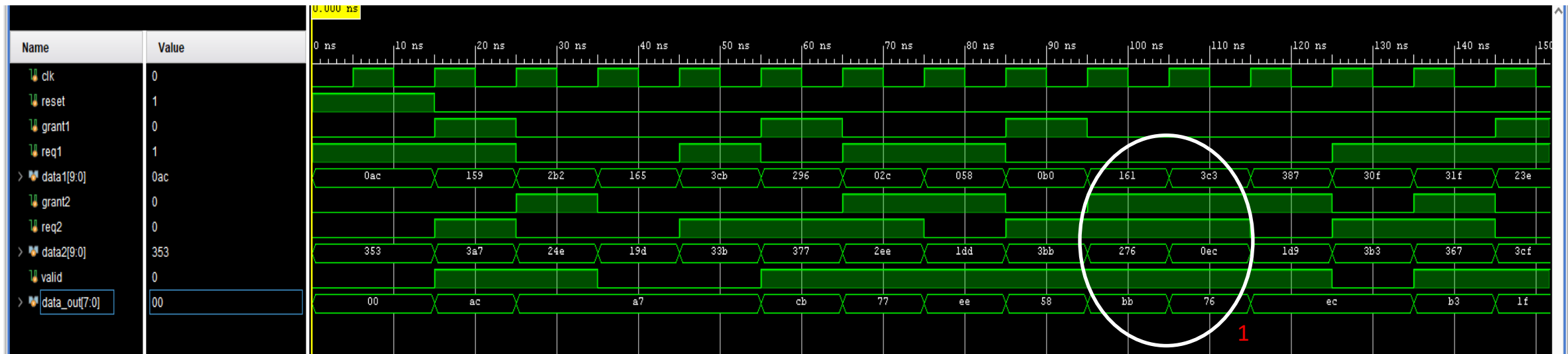
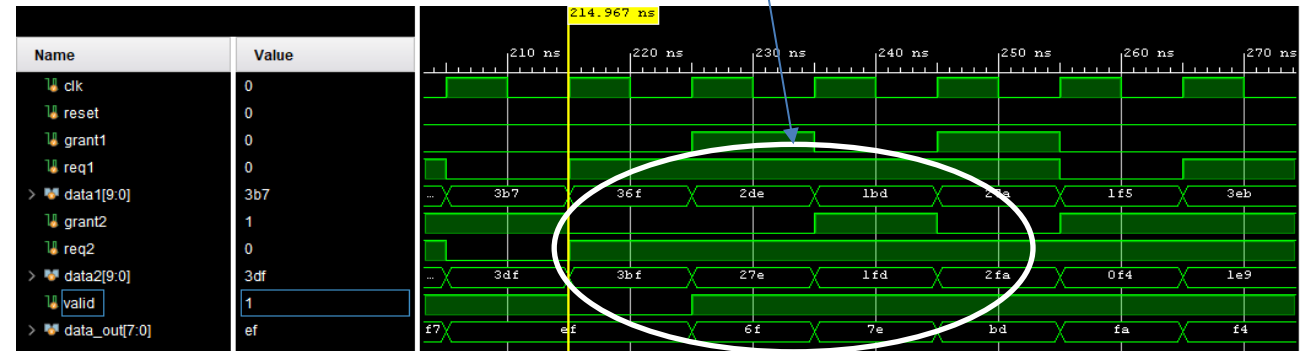
- 1. Output data is as expected, since req1 is high.
- 2. Valid for 0xd2 is held at high for an extra clock cycle during the first-round robin.
- 3. As per the design constraint, once link 1 wins, link 2 should win on the next clash of both req and priority bits



IP Enc and Dec with Mini Router in Simulation

- No specification on random or sequential data generated by the IPs
- 8-bit Linear Feedback Shift Register (LFSR) for pseudo-random data generation
- LFSR are cheap, only uses XORs and flip flops

```
req1 = 0 data1_pri = 3 data1_byte = 0xb7 grant1 = 0 | req2 = 0 data2_pri = 3 data2_byte = 0xdf grant2 = 1 | valid = 1 data_out = 0xef  
req1 = 1 data1_pri = 3 data1_byte = 0x6f grant1 = 0 | req2 = 1 data2_pri = 3 data2_byte = 0xbf grant2 = 0 | valid = 0 data_out = 0xef  
req1 = 1 data1_pri = 2 data1_byte = 0xde grant1 = 1 | req2 = 1 data2_pri = 2 data2_byte = 0x7e grant2 = 0 | valid = 1 data_out = 0x6f  
req1 = 1 data1_pri = 1 data1_byte = 0xbd grant1 = 0 | req2 = 1 data2_pri = 1 data2_byte = 0xfd grant2 = 1 | valid = 1 data_out = 0x7e
```



How to Implement the XTEA into the Cryptography System

- Mini router outputs data with 8 bits worth of payload
- Once the buffer reaches 32 bits, the double word can be sent into the XTEA algorithm
- The grant signal can be concatenated with the router output data and then used for a mux
- XTEA algorithm does both encryption and decryption on the data
- Based on the grant, the MUX outputs the correct operation done on the input data